Nubra · Product Management Intern Assignment

Habit Ledger

A post-trade review that only names habits it can defend

Problem 03 · Post-trade review

Working application

https://habit-ledger-xi.vercel.app

Source code, tests and documents

https://github.com/vinayakarya02/habit-ledger

Contents of this submission: 1. Product one-pager · 2. Prompt log · 3. Evaluation report · 4. Working application

Submitted 23 October 2026. All trader data in the app is synthetic; the prototype is not connected to Nubra or any broker and gives no investment advice.

Contents

1. Product one-pager 3
My take on the problem 3
Why Nubra should solve it 3
Solution space 3
What I built 3
In scope 4
Out of scope 4
What would prove me wrong 4
2. Prompt log 5
1. Research Nubra, generate concepts, choose one · 18:04 5
2. Refine Habit Ledger and build it · 18:20 8
3. Deploy to Vercel · 18:55 9
4. Upgrade to v2: nine modules, tests, GitHub, deploy · 19:04 9
5. Reply about the repository · 19:05 15
6. Repository choice · 19:06 15
7. Feature guide (Word document) · 22:25 15
8. Final submission PDF · 22:41 15
9. Prompt log scope · 22:42 17
3. Evaluation report 18
1. Ship criteria 18
2. Automated suites 18
3. Changes to existing tests (all deliberate) 19
4. Adversarial review (real, multi-agent) 20
5. Manual browser QA (23 Oct 2026, local preview, Chromium) 22
6. What these evaluations do *not* establish 22
7. AI in the product 23
4. Working application 24
Deployment check (23 Oct 2026) 24
Try it in two minutes 24
Supporting documents in the repository 24

1. Product one-pager

Habit Ledger: a post-trade review that only names habits it can defend

Problem 03 · Post-trade review · App: https://habit-ledger-xi.vercel.app · Code: https://github.com/vinayakarya02/habit-ledger

My take on the problem

"Turn a month of order history into the two or three habits that are costing them money" contains three tests that most post-trade tools fail:

- **"Costing"** needs a counterfactual. About 91% of individual F&O traders lost money in FY25 (SEBI, July 2025), so nearly every bucket of their trades loses. Subtracting losing trades always "finds" a cost. The honest question is how much *worse* these trades did than the trader's usual trade.
- **"Habits"** means repetition. A month is 30–200 trades, and one blow-up can impersonate a pattern. A credible review must be willing to say "not enough evidence".
- **"Them"** means a personal rule the trader can act on, not generic tips.

A later SEBI study found loss rates *rise* with experience. Experience alone doesn't correct behaviour, so the review must close a loop.

Why Nubra should solve it

Nubra holds both ends of that loop: the tradebook and P&L reports in its public API, and Flexi and strategy orders with stop-losses and whole-strategy exits. Nothing public connects a trader's history to *which* lever they need. Competing reviews (e.g. Upstox Trading Insights' expiry and call/put buckets) describe where money went. They don't test whether a behaviour is real, price it, or turn it into a rule. Nubra's multi-leg users also show a habit generic tools miss: closing the hedge and keeping the naked short.

Solution space

I compared descriptive dashboards, AI-written journals, rule-break ledgers (cold start), market-price replays (need intraday data, invite hindsight), and **evidence-gated habit detection with a guardrail loop**. Only the last is personal, falsifiable and actionable from a tradebook alone.

What I built

Five behaviours are checked, using only what the trader knew when deciding:

- re-entering within 15 minutes of a loss;
- adding to a losing option buy;
- removing a spread's hedge;
- sizing up after two wins;
- trading past the 5th trade of a day.

Cost is the excess loss versus the trader's own usual trade, capped so it never counts forgone profit. A behaviour becomes a **habit** only if:

- it occurs at least 4 times on at least 3 days;
- it is material;
- it still matters without its single worst trade;
- the period has 20 or more trades.

A disciplined trader gets **zero** warnings.

Nine connected modules make that reviewable:

- an **Overview** with one next action;
- **Habits**, with the reason for each classification;

- an **Evidence inspector** showing the trades, baseline, arithmetic, leave-one-out, and a re-gate when you exclude trades;
- a **Trade explorer**, where missing values show as "—", never ₹0;
- **Guardrails**, each labelled as a documented Nubra order control or a proposed capability, with compliance measured from fills;
- **Compare periods**, using rates per chance and an exact test, so trading less never counts as improving;
- **CSV import** with column mapping, validation and duplicate checks; data stays in the browser;
- a **Scenario lab**;
- a **Methodology** page.

140+ automated tests check planted behaviours against ground truth, recompute costs independently, test threshold boundaries and false-positive traps, and render every view across ~90 states. Bugs found by adversarial review became tests.

In scope

- Closed option trades (single- and multi-leg) from CSV or synthetic sample traders, reviewed month by month.
- No AI at runtime, by design: deterministic maths keeps every number auditable.

Out of scope

- Live Nubra connection, credentials or order placement.
- Nubra's own tradebook format, whose fields aren't public.
- Market-price counterfactuals.
- Equity, futures and tax.
- Per-user thresholds, which would invite fishing for patterns.
- Any trade recommendation.

What would prove me wrong

- Traders don't adopt guardrails.
- Guarded habits' rates fall no more than unguarded ones'.
- Traders dispute the evidence.

A launch would track all three, plus the share of zero-habit reviews (near 0% means the gates are loose) and order counts after review, which must not rise.

2. Prompt log

Every prompt I gave Claude Code (desktop app, Claude Opus 5.5) for this project, quoted word for word in the order I sent them, on 23 Oct 2026 (times in IST).

1. Research Nubra, generate concepts, choose one · 18:04

> @"C:\Users\aryav\Downloads\Nubra Product Intern Assignment (4).pdf"
>
> Role
> Act as a combination of:
>
> * A top-tier fintech Product Director who has shipped trading products
> * A startup founder who identifies non-obvious product opportunities
> * A skeptical product interviewer evaluating Product Management Intern candidates
> * A UX researcher, systems thinker, and AI product architect
> * A product engineer who can turn a validated concept into a working prototype
>
> Your goal is to help me create an outstanding submission for Nubra's Product Management Intern take-home assignment. Do not jump straight into building an app.
> Context
> Company: Nubra, operated by Zanskar Securities.
> Official website: https://nubra.io/
> Product/API documentation: https://nubra.io/products/api/docs/index.html
> Assignment: I have attached the original two-page PDF titled "Nubra Product Management Intern Assignment."
> The assignment asks me to select ONE of four problems:
>
> 1. Stock Fundamental Research Assistant
> 2. Position Risk Assistant
> 3. Post-trade Review
> 4. First F&O Trade
>
> Required deliverables:
>
> 1. A 300–700-word product one-pager explaining the problem, why Nubra should solve it, solution space, scope, and out of scope.
> 2. Every prompt used to build the product, including unsuccessful prompts.
> 3. Evaluation cases and the criteria used to decide whether the output was good enough.
> 4. A working application link, built using Lovable, Bolt, Replit, or another suitable tool.
>
> Time budget: 4–6 hours.
> Deadline: Within two days.
> The assignment explicitly says domain knowledge is not required and that problem framing matters more than visual polish.
> My Objective
> I do NOT want a generic assignment solution.
> I want a submission that demonstrates unusually strong product judgment, original thinking, customer empathy, technical feasibility, measurable evaluation, and a clear understanding of how a trading product works.
> Assume the evaluators are experienced product builders. They may already know the obvious solutions and may have already built features resembling them.
> Do not assume we know Nubra's internal roadmap. Verify public information and label unverified capabilities as hypotheses.
> Phase 1: Research Nubra
> Research its public website, product pages, help documentation, app listings, API documentation, release notes, and other credible public sources.
> Determine:
>
> * What Nubra already offers
> * Which user types it serves
> * The problems its current features appear designed to solve
> * What its existing capabilities imply for each of the four assignment options

* Where an incremental clone would add little value
* What genuine unmet needs may remain

Create a table with:
Existing capability | Evidence and source | User problem addressed | Potential gap | Confidence level.
Cite sources and distinguish verified facts from inference.
Do not invent internal features, customer interviews, usage data, business metrics, or product gaps.
Phase 2: Find the Non-Obvious Opportunity
Generate at least 12 genuinely different product concepts across all four assignment choices.
Do not merely rename dashboards, add an AI chatbot, summarize data, or generate generic alerts.
Explore ideas involving:

* Decision quality rather than information quantity
* User intent versus actual behavior
* Context-aware workflows
* Explainability and trust
* Behavioral patterns and repeated mistakes
* Scenario rehearsal and counterfactuals
* Risk communication and comprehension
* Missing information and uncertainty
* Workflow friction and time-sensitive decisions
* Evaluation and feedback loops

These are exploration directions, not requirements. Do not force every idea into them.
For every concept, explain:

1. The precise target user
2. The specific moment the problem occurs
3. The user's current workaround
4. Why the problem matters
5. The core insight
6. The product intervention
7. How it differs from known Nubra features
8. What evidence would validate or reject the idea
9. Whether a credible prototype can be built in 4–6 hours
10. The main risks and limitations

Phase 3: Think Like a Skeptical Evaluator
Compare all four assignment options. Do not automatically choose Position Risk Assistant just because it sounds technically difficult.
Evaluate concepts on:

* Originality and differentiation: 20%
* Strength of problem framing: 20%
* Demonstrable user value: 15%
* Feasibility within the time limit: 15%
* Quality of measurable evaluation: 15%
* Fit with Nubra's business and product: 10%
* Clarity and usability: 5%

Score each concept out of 100 and explain the scores.
Then challenge the top three ideas:

* Could Nubra already have this?
* Is this just a feature disguised as a product?
* Is the user problem sufficiently painful?
* Would an experienced PM find this obvious?
* Does the idea depend on unavailable data or APIs?
* Is AI genuinely useful, or would deterministic logic be better?
* Can a five-minute demonstration make the value immediately obvious?
* What would cause the submission to fail?

Revise the ranking after this critique.
Phase 4: Choose One Winning Concept
Select ONE concept only.
The winner should combine:

* A non-obvious, defensible insight
* A narrowly defined but meaningful user problem
* A strong fit with the assignment
* A product experience that can be demonstrated end to end
* Clear evaluation criteria
* A feasible implementation without privileged company access

Do not overbuild. A small, original, well-tested product is preferable to a large collection of generic features.
Explain why the winner is stronger than the other finalists.
Phase 5: Define the Product
Produce:

1. Product name and one-sentence value proposition
2. Target user and specific use case
3. Current workflow and its failure point
4. Core hypothesis
5. Primary user journey
6. Key screen-by-screen experience
7. MVP scope and explicit exclusions
8. Differentiation from Nubra's verified public capabilities
9. Success metric and guardrail metrics
10. Three assumptions that must be validated
11. A realistic demo scenario
12. A five-minute demo script

Prioritize user decisions and outcomes over the number of features.
Phase 6: Design Rigorous Evaluations
Create at least 12 test cases covering:

* Normal scenarios
* High-risk edge cases
* Incorrect or missing inputs
* Contradictory user intent
* False positives and unnecessary alerts
* False negatives and missed risks
* Numerical correctness
* Explainability
* User comprehension
* Uncertainty and limitations

For each case provide:
Input, expected behavior, pass/fail criteria, severity, and what the test reveals.
Where numerical calculations are involved, use deterministic calculations and independently check the expected answers.
Where AI is used, identify which parts require structured outputs, validation, and fallback behavior.
Never claim that the product is safe or accurate merely because the demo works. Define the limits of what the evaluation establishes.
Phase 7: Implementation Plan
Recommend the fastest suitable way to build the product, choosing between Lovable, Bolt, Replit, or another practical tool.
Design a polished, credible, interactive prototype with:

* A clear first-use experience
* Realistic sample data
* At least one complete end-to-end workflow
* Functional controls, not decorative buttons
* Useful empty, loading, error, and edge states where appropriate
* Transparent labels for simulated data
* No fabricated live market access

* No requirement for Nubra credentials or private APIs

Prefer deterministic calculations for financial metrics. Use AI only when it contributes a genuine capability.
Give me:

1. The proposed architecture
2. The data model and sample data
3. The complete app-building prompt
4. Follow-up prompts to fix likely implementation failures
5. A manual QA checklist
6. Deployment and access-verification steps

Do not claim to have built or deployed anything unless it actually has been done.
Phase 8: Final Submission
Produce the final four deliverables required by Nubra:
A. A polished 300–700-word one-pager, ready to submit.
B. A chronological prompt log, including unsuccessful prompts and actual outcomes. Clearly distinguish prompts that have genuinely been used from prompts merely proposed for future use.
C. An evaluation document with test cases, pass criteria, actual results where tests have been run, and unresolved limitations. Never fabricate test results.
D. A working app link only after the app has actually been deployed and access has been verified. If deployment is not possible, explain exactly what remains to be done.
Working Rules

* Research first, ideate second, select third, build last.
* Be critical, concrete, and evidence-driven.
* Avoid generic startup jargon and feature lists.
* Do not optimize for visual polish at the expense of product insight.
* Do not claim novelty without comparing against public product capabilities.
* Do not fabricate customer evidence, company strategy, metrics, or evaluation results.
* Do not give personalized investment recommendations or encourage risky trading behavior.
* Make assumptions explicit.
* Stay within the assignment's 4–6-hour build budget.
* If information is missing, make reasonable reversible assumptions rather than blocking the entire process.
* Present your findings in stages, starting with research, the 12 concepts, the scoring matrix, and the winning recommendation.
Do not begin implementation until the concept has been selected and its rationale is clear.

Start by examining the attached assignment and researching Nubra's public product. Then execute the phases in order.

2. Refine Habit Ledger and build it · 18:20

We have already completed the research, evaluated 14 concepts, compared competitors, and selected Habit Ledger as the winning concept for Nubra's Product Management Intern assignment.
I do NOT want to restart the research, brainstorm new products, or select another concept.
Our final direction is Habit Ledger: a product that identifies repeated trading behaviors associated with losses, validates them using evidence, estimates their financial impact honestly, maps them to relevant guardrails, and measures behavioral change over time.
Your task now is to refine this existing concept only where necessary and move toward execution.

1. Identify the most important weaknesses in the current Habit Ledger proposal.
2. Make targeted improvements without changing the core concept.
3. Define the final MVP that can be built within the assignment's 4–6-hour time budget.
4. Design the complete user journey and screens.
5. Define the deterministic behavior-detection logic, financial calculations, and evidence thresholds.
6. Create realistic synthetic trade histories, including a trader with repeated harmful behaviors and a disciplined trader who should receive no unsupported warnings.
7. Write the complete app-building prompt for Claude Code, Lovable, Bolt, or the most suitable available tool.
8. Define at least 12 evaluation cases with expected outputs and pass/fail criteria.
9. Prepare the 300–700-word submission one-pager.
10. Maintain an honest chronological prompt log and document actual evaluation results without fabricating them.

> The product must stand out through evidence-backed insights, honest uncertainty, behavior-specific guardrails, and a measurable feedback loop—not through generic AI features or unnecessary complexity.
> Do not repeat the research or reopen product selection. Do not build a different product. Refine Habit Ledger, then help me implement and test it.
> Start by presenting the final refined MVP and its screen-by-screen flow. Then provide the exact implementation prompt and proceed with building the product.

3. Deploy to Vercel · 18:55

> Yes, deploy Nubra/app to Vercel as a public static site, verify it loads logged-out, and add the link to the one-pager and prompt log.

4. Upgrade to v2: nine modules, tests, GitHub, deploy · 19:04

> Mission: Transform Habit Ledger into a Standout Fintech Product
> You are now acting as a senior product designer, staff frontend engineer, fintech product manager, QA engineer, and deployment engineer.
> We have already researched Nubra, evaluated 14 product concepts, selected Problem 03 (Post-trade Review), built Habit Ledger, run automated tests, and deployed an initial prototype.
> Do not restart research or replace the core product concept. Upgrade the existing product substantially.
> Current deployed app:
> https://habit-ledger-xi.vercel.app
> Current project: `habit-ledger`
> The existing codebase, tests, evaluation documents, one-pager, and prompt log are the starting point. Inspect the actual files before making changes. Do not assume the current deployment and local source are identical without checking.
> 1. Product vision
> Habit Ledger should become a sophisticated, interactive behavioral intelligence product for active options traders.
> The original problem is:
> "Most retail traders never review their own trades. Turn a month of order history into the two or three habits that are costing them money."
> Our product helps traders:
>
> * Understand repeated behavioral patterns in their trade history.
> * Inspect the actual trades supporting every finding.
> * Understand the limitations of financial attribution.
> * Choose appropriate personal guardrails.
> * Track whether their behavior changes over time.
> * Learn from previous decisions without receiving generic trading advice.
>
> The product must remain focused on post-trade review. Do not turn it into a generic brokerage dashboard, an AI chatbot, or a live trading terminal.
> 2. First: audit the existing implementation
> Before changing anything:
>
> 1. Inspect the complete project structure.
> 2. Identify the current UI, data model, rules engine, calculations, tests, and deployment configuration.
> 3. Run the existing automated test suite.
> 4. Record the current behavior and any failing tests.
> 5. Inspect the live deployment and compare it with the local source.
> 6. Preserve all existing working features and the original assignment requirements.
>
> Do not delete the existing implementation and rebuild a generic template.
> Create a short implementation plan, then execute it. Do not stop after producing the plan.
> 3. Upgrade the product experience
> Design a polished, cohesive, production-quality prototype.
> Visual direction:
>
> * Premium fintech analytics product.
> * Restrained dark/light theme support.
> * Clear information hierarchy and excellent typography.

* Consistent spacing, grid, cards, tables, charts, and interaction patterns.
* Responsive desktop, tablet, and mobile layouts.
* Subtle animations only where they improve understanding.
* Clear typography for financial figures and percentage changes.
* Accessible contrast, keyboard navigation, and meaningful focus states.

Avoid:

* Generic AI-generated dashboard aesthetics.
* Excessive gradients, glowing borders, decorative charts, and unnecessary cards.
* Fake live-market tickers.
* Arbitrary financial numbers.
* Buttons that do nothing.
* Empty pages disguised as features.
* Unsupported claims about profitability or prediction accuracy.

Use a coherent visual design system rather than styling every screen independently.
4. Build these product modules
Prioritize the features that strengthen the core behavioral-review loop. Implement them as working interactions, not static mockups.
A. Overview and monthly review
Create an overview that answers:

* What happened during this review period?
* Which behaviors have sufficient evidence?
* Which behaviors deserve attention first?
* What changed compared with the previous period?
* What action can the trader take next?

Include:

* Review-period selector.
* Total trades analyzed and data-quality status.
* Realized P&L and relevant trade statistics where supported by the data.
* Confirmed habits and near misses.
* Behavior-change indicators.
* A clear next action.

Every metric must have a defined calculation and appropriate context. Do not add a metric merely because it looks good in a card.
B. Habit intelligence
Provide detailed analysis of patterns such as:

* Re-entering shortly after a loss.
* Adding to losing long-option positions.
* Removing a protective hedge.
* Increasing position size after wins.
* Trading beyond a self-defined daily limit.

For each finding, show:

* A clear name and explanation.
* Occurrence count and relevant denominator.
* Supporting dates and trades.
* Evidence threshold results.
* Financial impact calculation and assumptions.
* Whether the pattern is confirmed, a near miss, or insufficiently supported.
* The reason for the classification.

Allow the user to filter, sort, and inspect the evidence.
Do not force every trader to have a confirmed habit.

C. Trade explorer
Add a functional trade-history explorer with:

* Search and filters.
* Date range.
* Instrument and trade type where available.
* Realized P&L.
* Entry and exit information where available.
* Associated behavior labels.
* A detail view for each selected trade.

Use the actual data model. Clearly distinguish missing values from zero.
Do not invent execution details or imply access to real Nubra accounts.
D. Evidence and calculation inspector
This should be one of the strongest differentiators.
For each habit, let users inspect:

* The exact trades included.
* Why each trade qualified.
* The relevant comparison baseline.
* How the financial estimate was calculated.
* What happens when the worst trade is removed.
* Which evidence thresholds passed or failed.
* What cannot be concluded from the data.

Every displayed number must be traceable to the underlying data and calculation.
Do not present associated losses as guaranteed savings or proven causal effects.
E. Personal guardrail center
Create a workflow for turning a sufficiently supported behavior into a proposed intervention.
Examples:

* A cooldown after a loss.
* A self-defined daily trade limit.
* A warning when a protective hedge is removed.
* A position-size discipline rule.

Each intervention should include:

* The behavior it addresses.
* Why it is relevant.
* The proposed rule.
* Its current status.
* How compliance will be measured.
* A way to edit, disable, or remove it.

Distinguish publicly documented Nubra order controls from proposed new capabilities.
The prototype must never place real trades or modify an actual brokerage account.
F. Month-over-month behavior tracking
Build an interactive comparison between two review periods.
Track relevant behavioral rates rather than relying only on raw counts.
For example:

* Re-entries following losing trades divided by eligible losing trades.
* Hedge-removal incidents divided by relevant spread opportunities.
* Rule violations divided by opportunities to follow the rule.

Show:

* Previous and current values.
* Absolute and relative changes where meaningful.
* Sample sizes.

* Whether the change is improving, worsening, or inconclusive.
* An explanation of the verdict.

A reduction in trading volume alone must not automatically count as improvement.
Use comparable periods and avoid making strong claims when the sample is insufficient.
G. Data import and data quality
Make CSV upload a real, robust feature.
Include:

* A sample file download.
* Column mapping or clear schema requirements.
* Preview before import.
* Validation errors with actionable explanations.
* Missing-field and duplicate detection.
* A summary of accepted and rejected rows.
* A clear distinction between imported data and sample personas.

Never silently convert invalid financial values to zero.
Do not send uploaded trade data to an external AI service.
H. Interactive scenario lab
Add a scenario lab that lets users explore how a behavior rule would classify a synthetic trade history.
Users should be able to change relevant inputs and see the results update.
Include:

* A normal scenario.
* A repeated-behavior scenario.
* A borderline-evidence scenario.
* A clean, disciplined trader.
* A missing-data scenario.

Clearly label this as a simulation. Do not present the results as market forecasts or real trading recommendations.
I. Product methodology and transparency
Create a dedicated explanation page documenting:

* The behavior-detection rules.
* The evidence thresholds.
* Financial calculations.
* Baseline selection.
* Limitations.
* Known edge cases.
* How sample data is generated.
* What the evaluation tests do and do not establish.

Users should be able to understand why the product reached a conclusion.
5. Engineering requirements
Keep the existing working stack unless there is a compelling technical reason to change it.
Requirements:

* Separate UI components from the behavior engine.
* Use deterministic calculations for financial values.
* Maintain a single source of truth for derived metrics.
* Add input validation and sensible error handling.
* Keep behavior definitions explicit and testable.
* Ensure overlapping habits do not inflate aggregate totals.
* Preserve the minimum-data and insufficient-evidence rules.
* Use stable formatting for INR, percentages, dates, and missing values.
* Avoid hardcoded results in UI components.
* Ensure all interactive controls work.
* Prevent stale state after changing personas, filters, or imported files.
* Keep the application usable on small screens.

Do not add an LLM just to claim the product uses AI.

If a calculation cannot be made reliably from the available data, display an explicit limitation rather than fabricating a result.
6. Improve the evaluation suite
Preserve the existing 19 automated tests and expand or revise them where necessary.
Add tests for:

* Each major behavior rule.
* Threshold boundaries.
* Zero and negative values.
* Missing data.
* Duplicate trades.
* Overlapping behavior labels.
* Financial attribution.
* Period comparisons.
* Sample-size limitations.
* CSV import failures.
* Clean trader histories.
* UI interactions and state resets.

Use known expected results and independently verify important financial calculations.
Run the full suite after the changes. Fix regressions rather than weakening the tests.
Report the actual results, including any unresolved failures.
Do not claim that synthetic tests establish real-world effectiveness.
7. GitHub: push the code to my repository
Inspect the existing Git configuration and identify the repository currently associated with this project.
If a GitHub repository is already configured, verify that it is the intended destination.
If no repository is configured, ask me for the GitHub repository URL before pushing. Do not create a different repository without my approval.
Before pushing:

* Inspect the files being committed.
* Exclude `.env` files, API keys, tokens, private data, build artifacts, and unnecessary dependencies.
* Add or update `.gitignore`.
* Include the application source, tests, package manifest, README, and necessary configuration.
* Include setup instructions and the commands to run the tests.
* Document the product architecture and limitations.
* Ensure no secrets appear in the source or Git history.

Use clear, honest commit messages describing the actual work performed. Do not fabricate historical commits, fake development activity, or falsely attribute work to someone else.
Do not overwrite unrelated repository contents or force-push without approval.
If GitHub authentication is unavailable, explain the exact step I need to complete. Never ask me to paste a private token into chat.
8. Remove generic scaffolding and improve originality
Inspect the application for:

* Default starter-template branding.
* Placeholder text.
* Generic boilerplate copy.
* Unused components and dependencies.
* Dead links and buttons.
* Unexplained sample values.
* Contradictory metrics.
* Inconsistent design.
* Console warnings and errors.

Replace generic scaffolding with product-specific language, coherent design, meaningful interactions, and accurate explanations.
Do not insert fabricated claims about who built the app, fake testimonials, fake users, fake usage metrics, or misleading authorship information. The code and documentation should be professional, understandable, and accurately represent what the product does.
9. Deploy the final version
The current deployment is:

https://habit-ledger-xi.vercel.app
Preserve this URL if the existing Vercel project can be updated safely.
Deployment requirements:

1. Run all automated tests.
2. Run a production build.
3. Check the main user journey.
4. Verify mobile layout.
5. Check the browser console.
6. Deploy the updated version.
7. Confirm the public URL works without a Vercel login.
8. Test all major modules on the live deployment.
9. Verify that the deployed application corresponds to the latest source commit.

Do not expose private information or make uploaded user data publicly accessible.
If deployment requires a new credential or explicit approval, pause and ask me. Do not invent a successful deployment.
10. Submission documents
Update the existing submission documents to match the actual final implementation:

* `02_refined_mvp.md`
* `03_implementation_prompt.md`
* `04_one_pager.md`
* `05_evaluation.md`
* `06_prompt_log.md`

Preserve the original assignment requirements.
The one-pager must remain within 300–700 words.
The prompt log must distinguish prompts actually executed from prompts merely proposed.
The evaluation document must report actual test results and unresolved limitations.
The product description must match the shipped app. Do not claim features that were not implemented.
11. Final acceptance checklist
Do not declare the task complete until you have checked:

* Existing functionality preserved.
* Multiple meaningful product modules implemented.
* Major controls and workflows actually work.
* Financial figures are traceable and consistent.
* Evidence and uncertainty are clearly communicated.
* CSV import handles invalid data.
* Desktop and mobile experiences work.
* Existing and new tests pass.
* Production build succeeds.
* Code is pushed to the intended GitHub repository.
* No secrets or private data are committed.
* Public deployment works without login.
* Live behavior matches the source.
* Submission documents match the actual product.

Execution instructions
Start by inspecting the current repository, tests, and deployment configuration.
Then implement the improvements in logical stages, testing as you go. Do not merely return a plan, design description, or code snippets. Carry out the work using the tools and permissions available in the current environment.
If access or credentials are missing, tell me exactly what is needed and continue with everything else that can be completed safely.
At the end, provide:

1. A summary of meaningful product improvements.
2. The GitHub repository URL and latest commit hash.
3. The public deployment URL.
4. The test results.
5. The updated submission document locations.
6. Any remaining limitations or actions required from me.

> The objective is to make Habit Ledger feel like a thoughtfully designed fintech product with a defensible product insight—not a generic dashboard—while keeping the original assignment focused and credible.

5. Reply about the repository · 19:05

> bro that is created by you just before this prompt check there

6. Repository choice · 19:06

I answered a multiple-choice question from Claude ("No GitHub repo exists for this project. Where should I push the code?") by selecting:

> Create public repo (Recommended)

7. Feature guide (Word document) · 22:25

> Create a simple Word document (.docx) named `Habit_Ledger_Complete_Feature_Guide.docx` explaining every single thing in Habit Ledger v2.
> Cover all nine screens and every button, card, box, chart, filter, dropdown, label, popup, and interactive element.
> For each item, explain in simple language:
>
> 1. What it does
> 2. Why we built it
> 3. How it helps the user
>
> Keep explanations short and easy to understand. Include practical examples wherever helpful. Don't skip small UI elements or assume I already know anything about the product.
> Use the actual app and code to verify everything. Don't invent features or make unsupported claims.
> Make the document well-organized with headings and tables so I can easily find any feature and understand it before my product management interview.
> Save the `.docx` file in the repository, push only this document to the existing GitHub repository, and share the document link and download link with me. Do not modify the app or its existing functionality.

8. Final submission PDF · 22:41

> I want you to prepare the final submission for my Nubra Product Management Intern assignment. First, inspect the existing project, current app, and all submission documents. Do not rebuild the app or change its functionality.
> Live app: https://habit-ledger-xi.vercel.app
> GitHub: https://github.com/vinayakarya02/habit-ledger
> Part 1: Rewrite the prompt log properly
> The current prompt log reads too much like I gave Claude one huge assignment and let it do everything. I want a chronological, detailed prompt log that communicates the actual product-development journey.
> Organize it in this order:
>
> 1. Understanding the assignment: The four problem statements, requirements, constraints, and evaluation criteria.
> 2. Evaluating the four options: The problems, target users, potential value, risks, differentiation, and feasibility of each option.
> 3. Selecting the product: Why Habit Ledger was selected for the post-trade review problem, the alternatives considered, and the reasoning behind the decision.
> 4. Defining the solution: The user problem, product hypothesis, MVP scope, user journey, behavioral rules, evidence requirements, and success metrics.
> 5. Research and refinement with Claude: The actual prompts used to research Nubra, investigate competitors, challenge assumptions, and refine the product.
> 6. Building the prototype: The implementation prompts, initial product structure, sample data, core logic, and early evaluation.
> 7. Testing and iteration: Actual bugs, failed approaches, what each failure revealed, the corrective prompts or instructions, and the resulting fixes.
> 8. Expanding the product: The work behind the nine modules, interface improvements, and usability enhancements.
> 9. Final evaluation and deployment: Actual test results, deployment verification, remaining limitations, and final deliverables.

Critical requirements for the prompt log

* Include the actual prompts used, not just a summary of the work.
* Use the existing `06_prompt_log.md`, `03_implementation_prompt.md`, `prompts/` scripts, tests, and Git history where available to reconstruct the chronology.
* Break large prompts into clearly explained stages, but do not falsely represent sections of one prompt as separate prompts that were actually submitted independently.
* Where an exact prompt was not recorded, label it as a reconstructed summary, not a verbatim prompt.
* Clearly distinguish my decisions and responses, Claude-assisted research, Claude-generated implementation instructions, and automated agent workflows.
* Show how research and evaluation informed product decisions and how test failures led to changes.
* Include unsuccessful attempts and their actual outcomes.
* Do not invent personal research, manual work, independent decisions, prompts, test results, or development history that cannot be supported.
* Keep the tone natural and professional, demonstrating product thinking rather than trying to conceal AI assistance.

The purpose is to demonstrate the reasoning, iteration, and product judgment behind the project—not to claim that I personally wrote every line of code.
Part 2: Prepare all four required deliverables
Combine these four assignment deliverables into one polished PDF, in this order:
1. Product one-pager
Use the existing `04_one_pager.md`. Keep it within the assignment's required 300–700 words. Explain the problem, target user, why it matters, the proposed solution, differentiation, MVP scope, exclusions, and evaluation approach.
2. Chronological prompt log
Include the revised, evidence-based prompt log described above, with actual prompts and outcomes.
3. Evaluation report
Use `05_evaluation.md`. Include evaluation cases, expected behavior, pass/fail criteria, actual results where available, and limitations. Do not fabricate results or imply that synthetic testing proves real-world effectiveness.
4. Working application
Include the live application URL prominently and make it clickable:
https://habit-ledger-xi.vercel.app
Also include the GitHub repository link as supporting material:
https://github.com/vinayakarya02/habit-ledger
Part 3: PDF design and quality
Create one professional, submission-ready PDF named:
`Nubra_Product_Management_Assignment.pdf`
Requirements:

* A clean cover page with the product name, assignment title, live app link, and repository link.
* A table of contents and page numbers.
* Clear headings, readable typography, consistent spacing, and properly formatted tables.
* Clickable links for the app and GitHub repository.
* Keep the one-pager within its required word count.
* Keep the prompt log chronological and detailed enough to demonstrate the actual process.
* Avoid unnecessary filler, repeated content, exaggerated claims, or unsupported metrics.
* Make the document easy for a hiring team to scan while retaining the evidence needed to understand the work.

Part 4: Final verification
Before delivering the PDF:

1. Cross-check all content against the existing files and actual implementation.
2. Verify that the four required deliverables are all present.
3. Confirm the one-pager word count.
4. Confirm that prompt entries are honestly classified as actual or reconstructed.
5. Check that test results and limitations match the evaluation report.
6. Verify that both links work and are clickable in the PDF.
7. Render or inspect the generated PDF to check for broken tables, clipped text, blank pages, or formatting issues.
8. Do not modify the live app or invent any deployment status.
9. Keep the existing Markdown files intact unless a correction is necessary and justified.

> Save the final PDF in the project directory and provide me with its exact local path and a download link. Also provide a short summary of what it contains and any unresolved issues.
> The final result should present a coherent product journey: understanding the problem, comparing the four options, choosing Habit Ledger, refining the solution, using Claude to implement it, testing and fixing issues, and preparing the final submission.

9. Prompt log scope · 22:42

> prompt log should only have prompts thay you assume given by me nothing else

3. Evaluation report

Habit Ledger: Evaluations

How I decided it was good enough to ship (as a prototype):

1. It finds behaviours that are really there.
2. It stays silent when they aren't.
3. Every rupee figure can be recomputed by hand.
4. Bad input is rejected, never silently turned into ₹0.
5. Every control in the UI does what its label says.
6. Independent reviewers who are trying to break it can't find a confirmed problem that remains unaddressed.

Actual result (23 Oct 2026, Node v24.16.0):

- `npm test` passes **151 / 151** (0 fail, 0 todo, 0 skipped).
- `npm run smoke` renders 9 views × 90 states = **810 renders, 0 problems**.
- `npm run build` succeeds.
- The raw test output is saved in `app/tests/last-run.txt`.

All figures in this document come from actual runs.

1. Ship criteria

#	Criterion	Threshold	Result
S1	Detection against planted ground truth (5 behaviours × 6 sample trader-months, plus lab scenarios)	Detected set = planted set	Met (E01–E03, lab tests)
S2	The disciplined trader gets no confirmed habit	0 in both months	Met (E04, U-tests)
S3	Each near miss fails for the intended reason	3 / 3	Met (E05)
S4	Costs equal an independent recomputation from raw fills	abs. error < ₹0.01	Met (E07, E08, engine rules)
S5	Invalid values are rejected with a reason, never coerced to 0	All malformed cases	Met (E10–E12, import tests, R13)
S6	No copy claims causation, forgone profit or advice	0 matches across all rendered states	Met (E17, U07)
S7	Verdicts follow the comparison rules, and trading less never counts as improving	All boundary cases	Met (E14, E15, compare tests)
S8	Every rendered control maps to a real action; no duplicate IDs; no leaked `NaN` or `undefined`	0 problems in 810 renders	Met (U01, smoke)
S9	Every control has an accessible name; one h1 per view; charts have text alternatives	All sampled states	Met (U03, U04)
S10	Confirmed adversarial-review findings addressed	57 / 57	56 fixed, 1 partly fixed (see §4.3)

2. Automated suites

File	Tests	What it covers	How expected values were derived
`evals.test.mjs`	19	The original v1 evals: planted vs detected, false-positive traps, cost arithmetic, gates, CSV failures, open positions, round trip, verdicts, de-duplication, honest copy, thin month, data realism	Planted manifest from the generator; a hand-worked spread; recomputation from raw fills

File	Tests	What it covers	How expected values were derived
`engine.rules.test.mjs`	34	Every detector at its exact threshold boundary, every gate boundary, baseline cap and thin baseline, zero and flat P&L, missing charges, overlap labels, period split, guardrail measurement with custom parameters, re-gate, the 3-habit limit	Small hand-built fixtures, with the expected figures worked out in comments
`import.test.mjs`	27	Reader (quotes, CRLF, BOM, `;` delimiter, line numbers), header aliases, mapping problems, timestamps (zones, no zone → IST, invalid dates, seconds ≥ 60), strict amounts (Indian and western grouping; decimal comma rejected), every row-level error, duplicates, the row limit, round trip	Literal inputs and outputs
`compare.test.mjs`	16	Fisher exact against hand counts (17/70, 1/6, 2/19, 1/26) and an exact reference; Wilson; every verdict boundary (minimum chances, ±50% / 25% relative, p on both sides of 0.2 and 0.05); "trading less is not improving"	Exact fractions
`lab.test.mjs`	10	Each preset's classification, determinism, gap outside the window, clamping, planted = detected	Generator manifest
`store.test.mjs`	23	Reducer purity and state resets (trader or period change resets filters, selection, exclusions and drafts); ignored bad input; guardrail flow; import flow; persistence sanitising; next-action priority; trade filters and paging; inspector sensitivity	Hand-checked counts from sample data
`ui.test.mjs`	9	Render matrix (no unknown actions, duplicate IDs or leaks); every effect implemented; accessible names; one h1; chart text alternatives; overview headline = engine headline; no stale data after switching trader; disclaimer present and banned phrases absent; overlays only when asked for; missing values shown as "—"	Rendered HTML
`review-fixes.test.mjs`	13	One regression test per engine or store fix from the integrated review (R01–R13, §4.2)	The reviewers' reproductions

The render matrix (`tests/helpers/render-matrix.mjs`) covers 90 states:

- 3 sample traders × 2 months × each habit in the inspector;
- trade filters, selections and dialogs;
- every import-wizard step, including a messy broker file and a thin file;
- every lab preset plus edge settings;
- three edge datasets: a habit cut only by the 3-habit limit, and two real overlaps.

3. Changes to existing tests (all deliberate)

Test	Change	Why
E15	Expected labels renamed: "Worse" → "Worsening", "Too little activity to judge" → "Inconclusive"	v2 verdict vocabulary. The same cases now go through a stricter rule (minimum 8 chances, exact test).
E17	Scans every file under `src/` instead of two files	The v1 `app.js` no longer exists. The scan is wider, not narrower.
E19	Days of Meera's hedge removals read from the ground-truth manifest	The sample calendar now skips NSE holidays, so the hard-coded dates moved.

Test	Change	Why
engine.rules "previous trade flat"	Re-entry chances 1 → 0	**Semantic fix** (review finding #1). A re-entry chance is now a *trade opened after a losing trade*. Before, chances counted losing trades while occurrences counted trades, so the rate could exceed 100%.
compare "costs floored at 0"	Chances label → "trades opened after a losing trade"	Same fix
lab and store "days" clamp	22 → 21	September 2026 has 21 NSE trading days
store selectDataset	Inspector opens on the new trader's top confirmed habit (H3 for Meera) instead of keeping H2	Review finding: switching trader showed an empty "not seen" page
store guardrail, persistence and import tests	Rule objects gained `settingsPeriod`, `savedAt` and `changedAt`; rejected rows gained `file`	New fields for the honest guardrail timeline and multi-file imports
store next-action	October with rules set → "review-change" (compare) instead of "wait"	Review finding: the month where the loop pays off was a dead end
store import notes	Timezone note counts 4 imported rows, not 6 rows read	Review finding: it counted rejected rows that never entered the data

No test was deleted, and no assertion was loosened to make a failure pass.

4. Adversarial review (real, multi-agent)

4.1 Module build workflow

Ten pipelines ran (nine modules and one test author), each through build, independent review and fix: 30 agents in total. The test author found **6 real bugs** and recorded them as `todo` tests instead of patching them:

1. A decimal comma "112,50" was read as ₹11,250.
2. A timestamp with seconds ≥ 60 became `NaN`.
3. The import notice double-counted excluded rows.
4. `initialState(null)` crashed.
5. The p ≥ 0.5 sentence was ungrammatical.
6. A 4th habit cut only by the 3-habit limit was called an "overlap".

All six were fixed, and the tests are now hard tests.

4.2 Integrated review

Five reviewers each took one lens:

- numbers and cross-module consistency;
- interaction and state;
- honesty and copy;
- accessibility and responsive design (in a browser);
- the product as a whole.

Two independent skeptics then tried to refute each finding.

	Count
Findings reported	66
Confirmed by both skeptics	**57** (1 critical, 24 major, 32 minor)
Split (one skeptic each way)	4
Rejected by both	5

The most important confirmed findings, and what changed:

- **Critical: the re-entry rate could exceed 100%.** Two trades opened after one loss both counted against a single "chance". This made compliance negative, gave a `NaN` interval and produced a false "strong Improving". A chance is now a trade opened after a losing trade, and the statistics functions reject impossible inputs (R01, R02).
- **Honesty: guardrail timeline.** Months already in the file were labelled "after the rule was set", implying the rule caused a change. Each rule now records when its settings were saved. A month is "after" only if its trades came later; otherwise it is labelled a what-if. Compare no longer says a rule was "broken" before it existed (R06).
- **Honesty: suggested stop levels.** The stop-loss snippet showed specific stop distances (0.70 / 0.68 × entry), and the adding-to-a-loser rule was labelled "documented" although no documented control blocks adding. The rule is now only the measured behaviour, labelled proposed. The snippet carries field names with placeholder values (R05).
- **Honesty:**
  - "Mostly the same trades" was used for 1 of 19 shared trades (R03);
  - limit cuts were called overlaps (R04);
  - "gross losses" actually included charges;
  - the headline said "vs your usual trade" for figures measured against ₹0 or a spread's maximum loss;
  - sample trades fell on NSE holidays (R11);
  - Kabir was described as using stops;
  - counts were not pluralised.
- **Product:**
  - "Import next month" was impossible: each import became a new trader. A file can now be added to an existing imported trader (R07).
  - The inspector had no way to set a guardrail.
  - Suggestive changes looked as certain as clear ones. They now read "Possibly improving", and behaviours that were never a habit are shown as context (R12).
  - The inspector opened on an empty habit (R10).
  - A clean trader was told to wait for a month already loaded.
- **Interaction and accessibility:**
  - slider drag broke after one step;
  - dialogs didn't trap focus;
  - Escape closed the dialog instead of the popover inside it;
  - re-renders reset scroll positions;
  - the skip link broke routing;
  - the live region was never written to;
  - the focus ring was hidden on active segments;
  - text contrast was slightly below AA, and input borders were 1.55:1;
  - popovers overflowed the viewport;
  - the chart's values were reachable by hover only;
  - the phone header took 43% of the screen.

Split findings:

- Implemented:
  - the loop-payoff next action (now "review-change");
  - the lab → importer handoff ("Open this month in the importer");
  - hiding another trader's data-quality panel while a file is checked.
- Not implemented: larger tap targets for the 16px "?" popover triggers. One skeptic rated this not-an-issue, the other minor.

Rejected by both skeptics: five findings, including that lot-cap compliance measures only the first fill (it measures entry size by design), and that the Overview leads with generic KPIs.

4.3 Fix verification

Engine, store, main and data fixes were made directly. Each has a regression test (R01–R13).

View fixes went to nine agents, one per module. An independent verifier per module checked every item against rendered text:

- all **9 / 9** were verified as resolved;
- four verifiers found small regressions introduced by the fixes (e.g. two live-region sources on one page, a missing space between sentences, copy that over-claimed), and a second pass fixed them.

Partly addressed: review finding #16. Fixing the unprompted re-render (the notice timer) is done. But any re-render still closes open `<details>` disclosures, because the app re-renders the whole tree.

5. Manual browser QA (23 Oct 2026, local preview, Chromium)

Check	Observed
Guardrail from the Overview "Next step" with a real click	Dialog opens with focus on the first setting; the page behind is `inert` with scroll locked; 8 Tab presses stay inside the dialog; arrow keys change the setting with a live preview; one click on Save closes the dialog, focus returns to the next-step button, and the live region announces "Saved … Nothing was sent to any broker."
Guardrails and Compare wording after saving	"Every period here is a what-if…"; Compare: "9 / 30 → 2 / 24 · Improving · … about 1 in 20 times by chance"; excess loss shown as context only
Trades: typing "BANKNIFTY" in search	Focus stays in the box through each re-render; "Showing 18 of 52 trades · net −₹7,120" announced
Trade drawer	Opens with focus on its heading; Escape closes it and returns focus to that row's Details button
Import of a messy broker CSV via the real file input	Header aliases mapped. Rejected with reasons: decimal comma, duplicate ID, FUT row, side "HOLD", negative quantity, blank price, non-numeric charge. The blank charge was imported and flagged. "1 position will be left out" is named before import. The new trader is thin, so the next step is "Add more history".
Lab slider: real mouse drag from 18 → 5	Value follows the drag to 5; the results recompute; focus stays on the slider
375px in dark mode, all 9 modules	0 horizontal overflow; the current tab is visible in the mobile tab bar; the drawer is a bottom sheet and the dialog fits
Console	No messages at all across the journey

Found in earlier browser QA and fixed: a 1-trade upload was reading as "no habits" (now "Too few trades to judge"), and the copy said "Some appeared" when nothing had.

6. What these evaluations do *not* establish

- **Real-world accuracy.** The data is synthetic and the behaviours were planted. Perfect detection shows the rules are implemented as defined. It does not show that real traders' habits look like this, or that the thresholds are right. Next step: consenting users' real tradebooks, with their own "that's me / that's not me" labels.
- **Causation.** Excess loss is an association; the UI says so wherever a cost appears.
- **Behaviour change.** October was also generated. The loop shows the *measurement* works, not that guardrails change behaviour, which would need a controlled comparison.
- **Statistical calibration.** Gates are a pragmatic small-sample filter. Testing five behaviours raises the chance of a false pattern. A null simulation by a reviewer showed a directional verdict about 10% of the time per behaviour per direction with the $p < 0.2$ bar, which is why such verdicts read "Possibly…".
- **Nubra integration.** Nubra's real tradebook schema isn't public. "Documented" labels refer to Nubra's public API docs, not verified app behaviour.

- **Accessibility.** Automated checks plus keyboard tests are not a full audit. Screen-reader testing on real assistive technology wasn't done.

7. AI in the product

There is none at runtime. Detection, costing, gates, statistics, verdicts and compliance are deterministic, so tests can assert exact values. Two places an LLM could earn its keep later, each with structured output, validation and a fallback:

1. **Mapping arbitrary broker CSVs.** Validate by zero row errors plus reconciliation with the broker's P&L. Fall back to the manual mapper, which exists today.
2. **Rephrasing habit cards.** Every number in the text must appear in the structured habit object. Fall back to template copy.

4. Working application

Live app: https://habit-ledger-xi.vercel.app

Repository: https://github.com/vinayakarya02/habit-ledger

The app opens on a synthetic sample trader with no sign-in. It runs entirely in the browser: no account, credentials or market data are needed, and imported CSV files are never uploaded.

Deployment check (23 Oct 2026)

Check	Result
Public URL without login	https://habit-ledger-xi.vercel.app returns HTTP 200
Deployed build	`/version.json` reports commit `2747181afe5f`, the app source commit on GitHub `main`
Hosting	Vercel static deployment (project `habit-ledger`)
Tests on that commit	`npm test` 151 / 151 pass; `npm run smoke` 810 renders, 0 problems; build succeeds

Try it in two minutes

1. Overview (sample trader Aarav, September 2026): 52 closed trades, net −₹1,01,510; two habits pass the evidence bar.
2. Open "Inspect evidence" on "Re-entering right after a loss": the 9 qualifying trades, the baseline, the arithmetic and the leave-one-out test.
3. Click "Set guardrail", choose a cool-off time and save. Nothing is sent to any broker.
4. Open "Compare periods": the re-entry rate falls from 9 of 30 in September to 2 of 24 in October (Improving), with sample sizes and 90% ranges.
5. Switch the trader to Kabir: no confirmed habits, only near misses with the reason each fell short.
6. Open "Import CSV" or "Scenario lab" to see invalid rows rejected with reasons, never read as ₹0.

Supporting documents in the repository

Document	What it contains
01_research_and_concept_selection.md	Research on Nubra and competitors, 14 concepts across the four problems, scoring and the choice of Habit Ledger
02_refined_mvp.md	Product specification as shipped: behaviours, gates, comparison, guardrails, modules
03_implementation_prompt.md	Build specification and the multi-agent workflow scripts used during the build
README.md	Architecture, how to run the app and tests, limitations